

Architecting reproducible science: Best Scientific Computing Practices

Software Engineering principles to improve your HPC

Fernando Garzon, August 2026

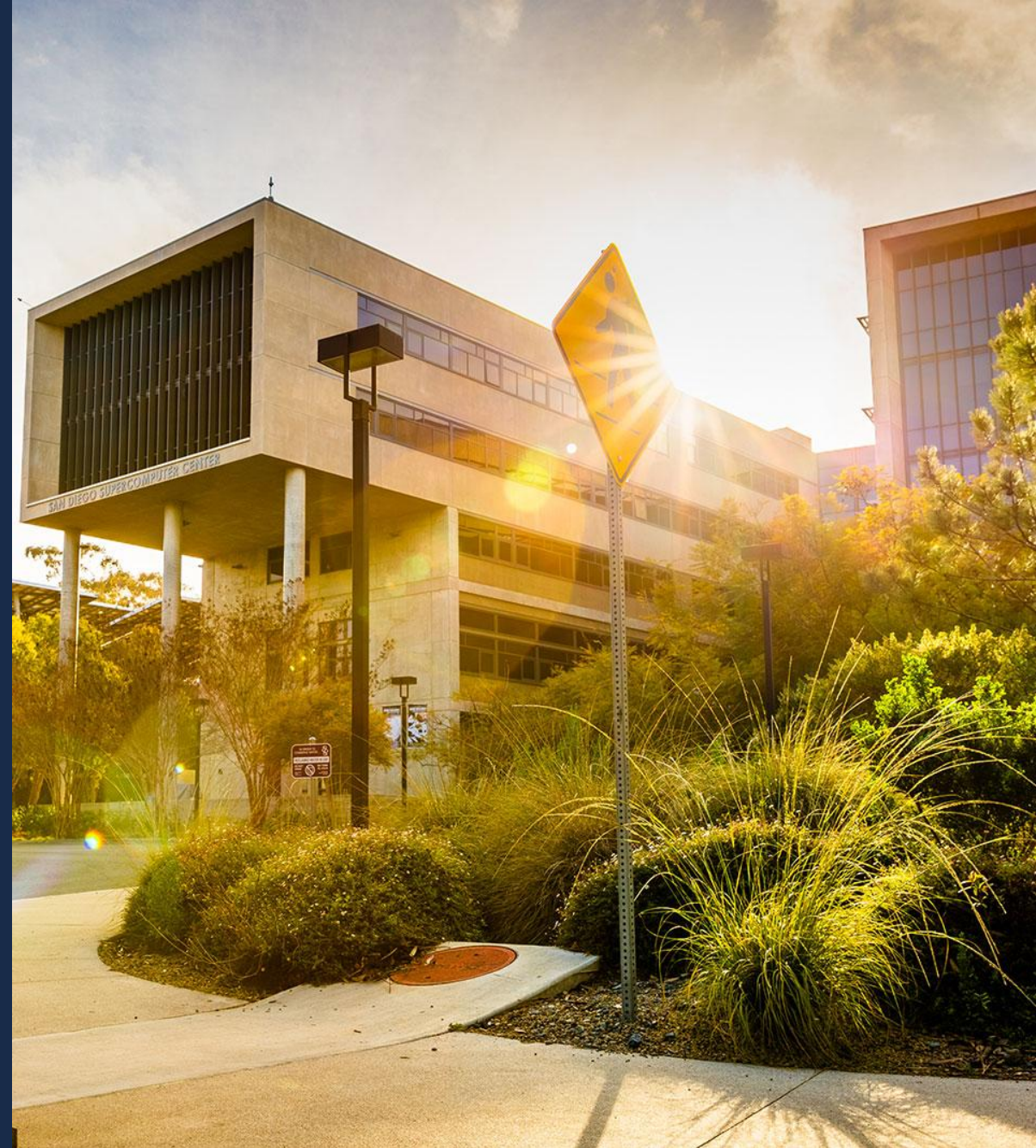
[linkedin.com/in/fgarzonm](https://www.linkedin.com/in/fgarzonm) - github.com/ofgarzon2662

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

Who's Fernando Garzon

- Software Engineer
- Computational & Data Scientist Research Specialist at SDSC
- Work on User Support for TSCC.
- Also work in Software Dev (SWD) projects within SDSC.
- Passionate about SWD, DevOps, Automation, Cloud Architecture, IaC and more.
- Automation Enthusiast.

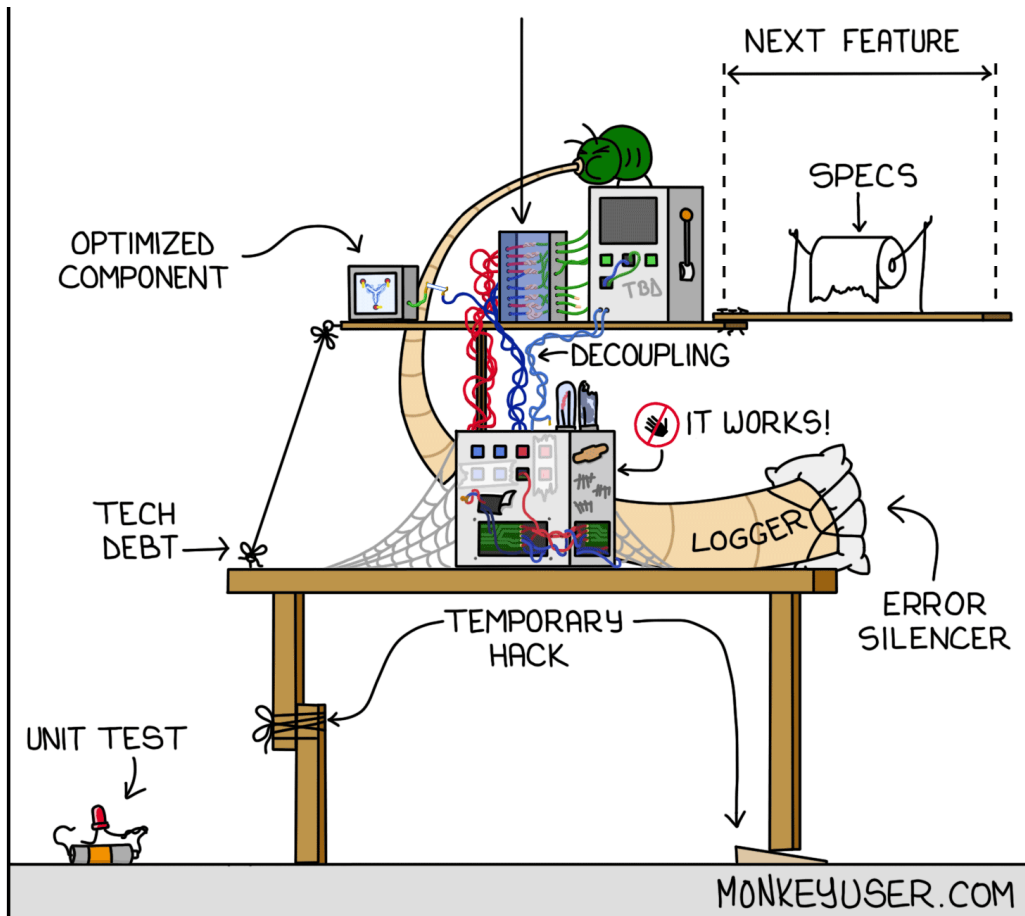




Outline

1. Intro: The un-reproducibility horror
2. Notebooks Don't Scale. Packages Do.
3. Let's Prod this.
4. Lights, Camera, GitHub Actions!
5. MLOps – Turning your Notebook into a Money-Printing Machine.
6. Final remarks

Intro: The un-reproducibility horror

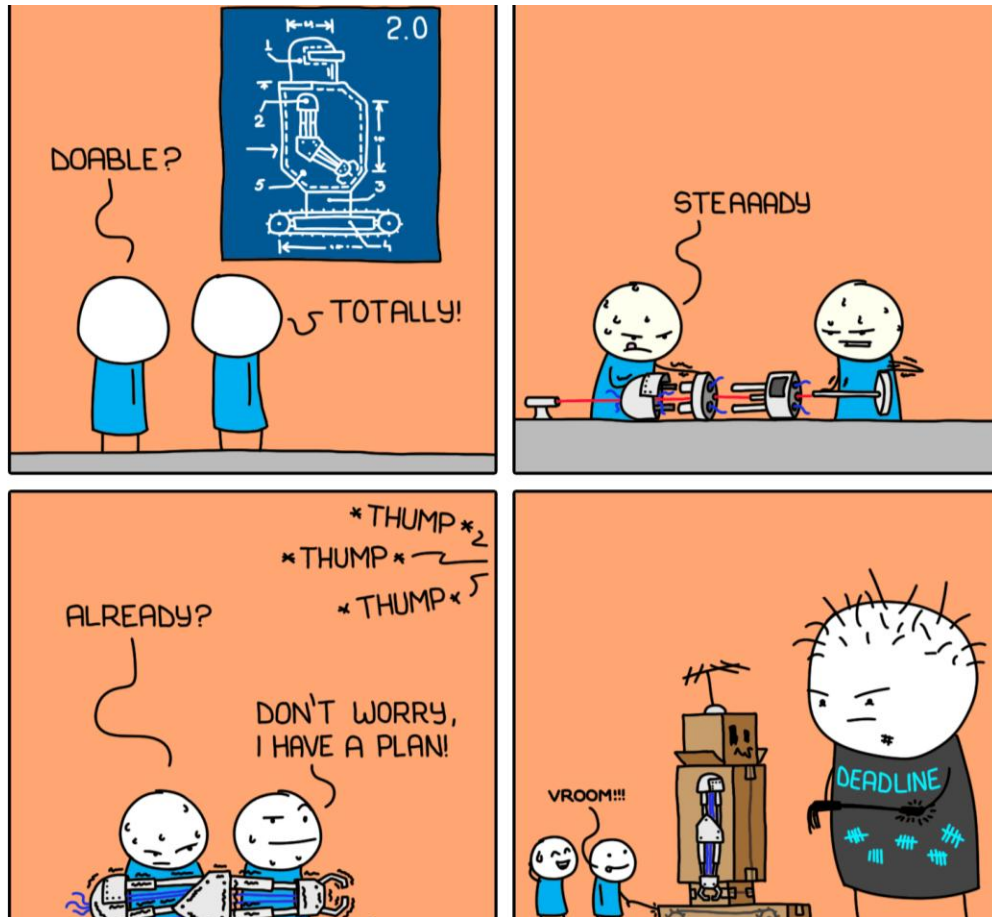


- Milena's model in the Jpy NB worked perfectly. It ingested and analyzed the data, generated the proper outputs, and most importantly, it promised a big balance in her and the stakeholders' bank accounts.
- The mess up: She forgot to run an apparently meaningless cell, one that optimized the routine to prevents memory overload. Fans roared and laptop froze.
- Results: Terrible. Stakeholders who know nothing about code and care less about it decided this isn't worth the headache.
- The lesson: There's just gotta be a better way...and there is.

Intro video:

[Link - Video](#)

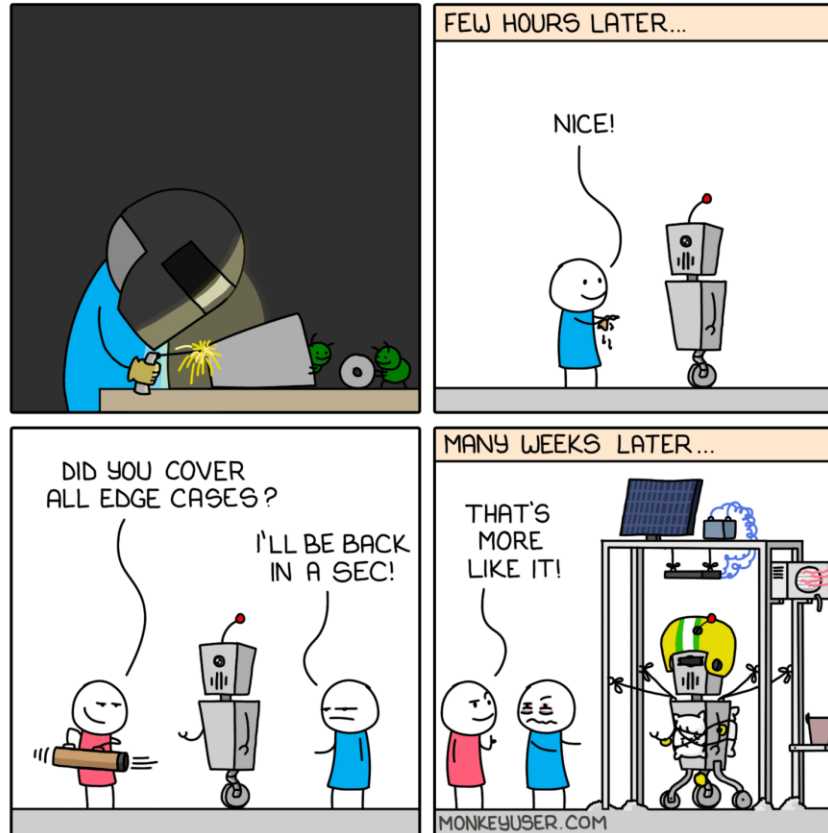
Notebooks Don't Scale. Packages Do.



- Well...not exactly, NBs can scale, just not as good as Modular Packages do.
- The key word is Reproducibility. Milena's issue is that she wasn't able to *reproduce* the great results in front of the stakeholders, all because of a missing cell execution.
- However, the word Reproducibility has many definitions.
- Here, I want to focus on:
 - Are the results **Repeatable**, is the workflow producing consistent and deterministic results?
 - Is the code **Portable**, can I launch it outside my laptop, say, on Expanse, where the computational capacity is greater and can I get more useful output faster?
 - Is it **Scalable**? Can I deploy my code in 1000s of servers at the same time and serve 1000s of users concurrently while ensuring minimum levels of performance, latency and security that are acceptable for the end users?

Notebooks Don't Scale. Packages Do.

EDGE CASES



- NBs can do all that and devs work on that every day: NBs can be **Scalable**.
- NBs are also amazing for exploratory work: You got an idea, you test it on the fly, you see instant results/plots/numbers just one 'shift' + 'enter' hit away.
- However, and for the *same* reason, if good coding practices aren't kept (and *most* people in academia don't), NBs can grow out of control quickly with messed up code all over the cells.
- And AI just facilitates the creation of big messed up code bases faster.
- So yeah, NBs are great, but hard to maintain and **Scale**.
- So, what's the other way?

Modular Python Packages

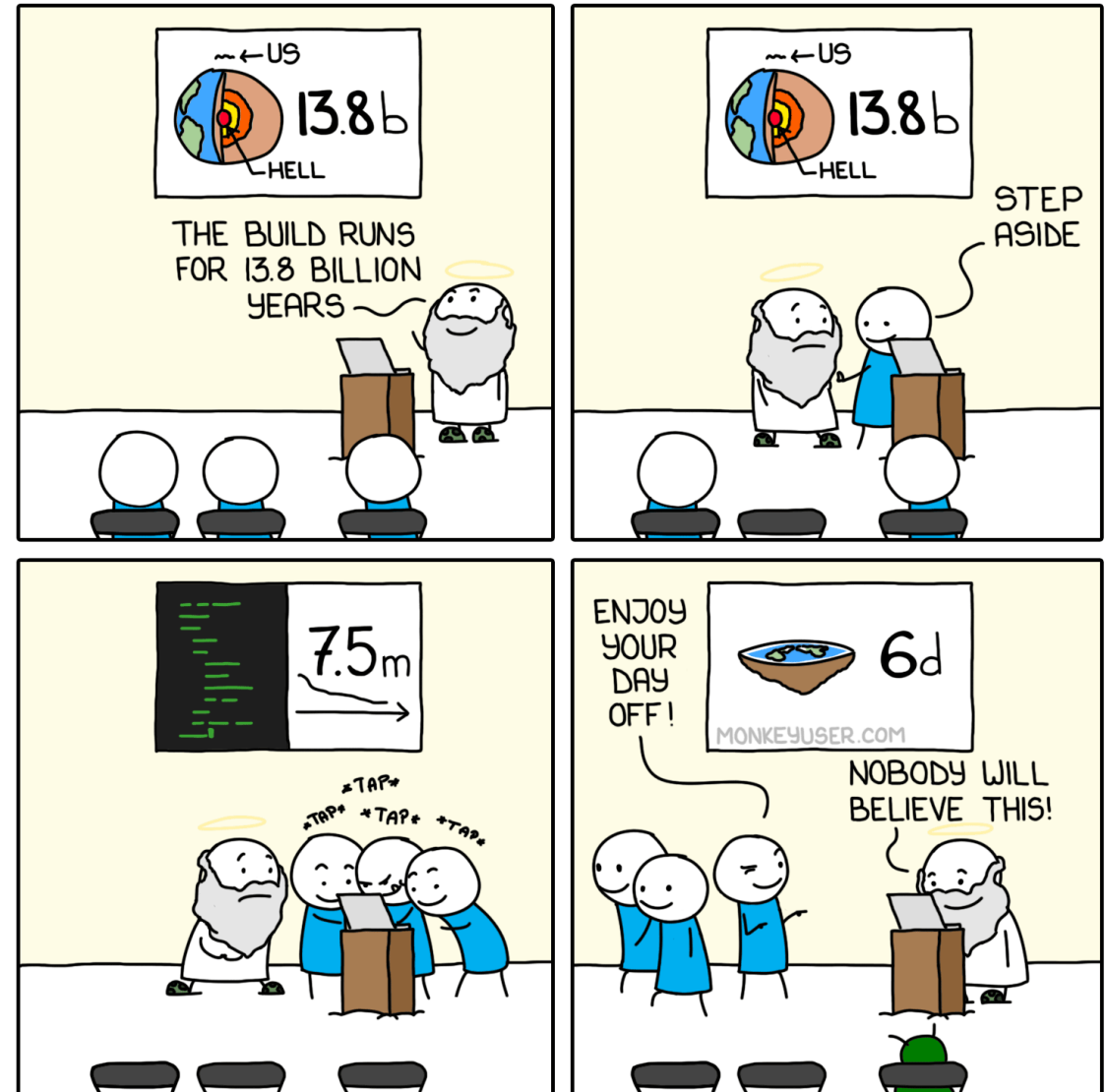
Let's Prod this

- *Modular Python Packages (MPyP)*: They Scale real nice.
- **IF** coded properly, they're small, manageable, Testable, easy to deploy, can run anywhere, and best of all, they're **Scalable**.
- They are like little bricks that you can handle easily with your own hands, instead of trying to carry the whole building around wherever you go.
- Therefore, the strategy is clear: We need to learn to migrate our Jupyter Notebooks to MPyPs

Follow the instructions in here:

https://github.com/ofgarzon2662/mnist_ae/blob/master/README.md

NEW WORLD



Let's Prod this

Prompt Engineering and AI assisted development. Use SOLID principles:

You are a senior Python engineer. Rewrite the file ``mnist_ae/mnist_training.py`` so that:

- *Each class/function has one clear responsibility (Single-Responsibility Principle).*
- *Related functionality is grouped into modules (e.g. `data`, `model`, `training`, `cli`).*
- *Internal helpers are made private (`_`prefix).*
- *No global execution at import-time; provide a ``main()`` entry point.*
- *Add type hints and docstrings.*

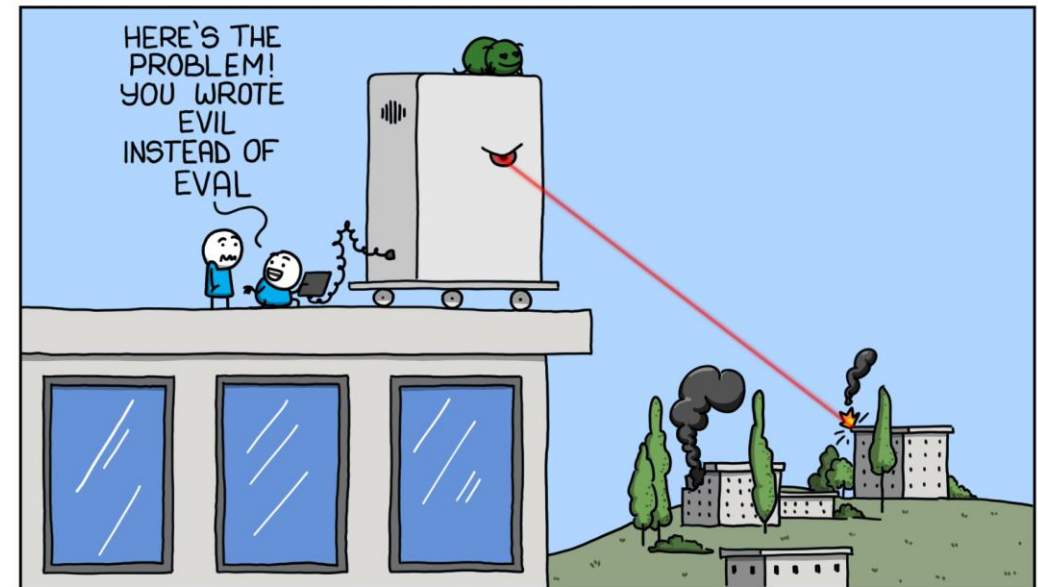
Return the full, refactored code as a valid Python package structure.

Follow the instructions in here:

https://github.com/ofgarzon2662/mnist_ae/blob/master/README.md

NATURAL LANGUAGE INSTRUCTIONS

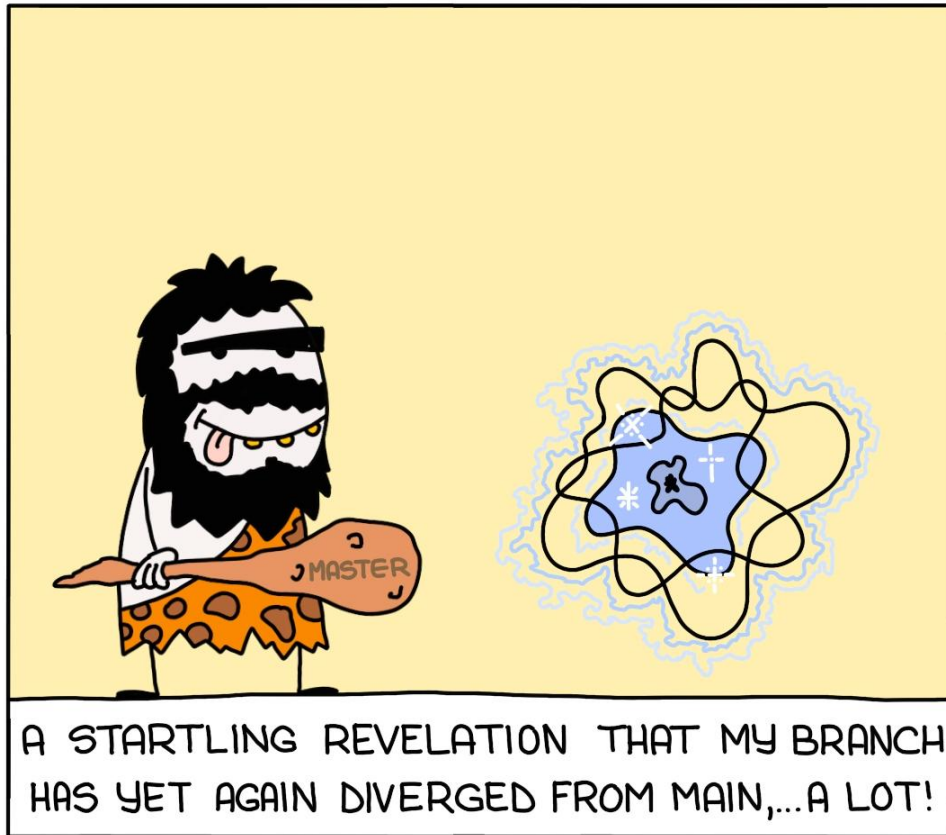
MONKEYUSER.COM



Lights, Camera, GitHub Actions!

RAMIFICATIONS

MONKEYUSER.COM



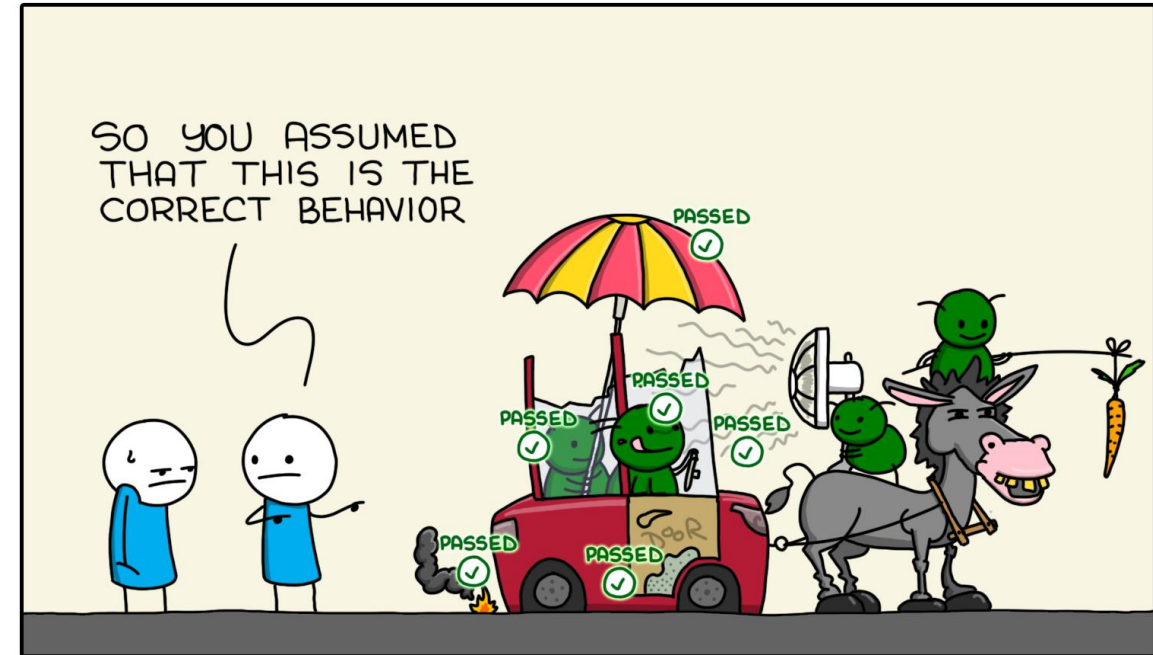
- DevOp\$ is a seriously promising profitable field of work. And it's been for the last 15 years.
- Combining DevOps with AI tools can make you super attractive to potential employers in high tech industries and HPC.
- DevOps is the art of automating deployments in a fast, controlled, and a reproducible manner.
- And, GitHub Actions are the 1st step in the DevOps stairs.
- Easy to implement, free or super cheap at worst, and they should be mandatory on every project you work on starting today.

Lights, Camera, GitHub Actions!

- The most basic automation one should always do is Unit Testing and code coverage.
- But you can extend your Github actions to cover Security, Linting, basic integration tests, basic deployment tests in dedicated VMs or envs, basic smoke tests.
- And, you can also implement Github actions to automate deployments of Python Packages.

UNIT TESTS

MONKEYUSER.COM

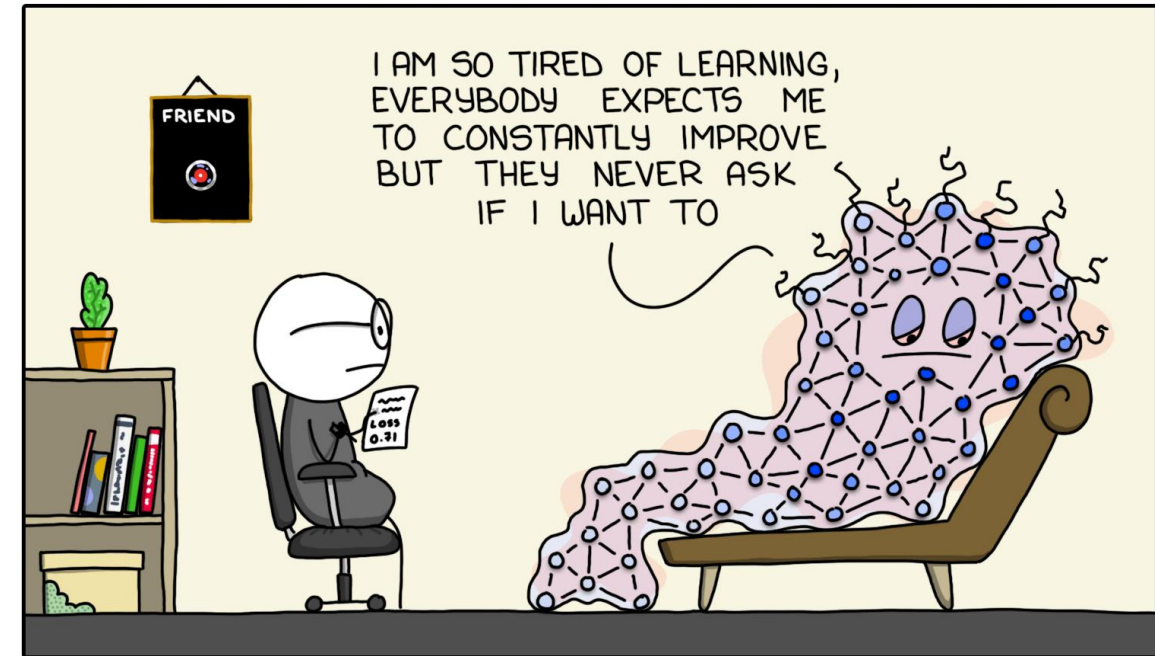


MLOps: Turning your Notebook into a Money-Printing Machine.

- MLOps is the combination of ML + DevOPS.
- That is: The automation of testing and deployment of ML Models through DevOps Practices.
- It's basically caring about the health and quality of your ML/AI Model in an efficient and automatic way.
- Imagine combining your ML expertise to forecast Housing prices with what you learned today...Can you forecast how much your income could potentially grow?

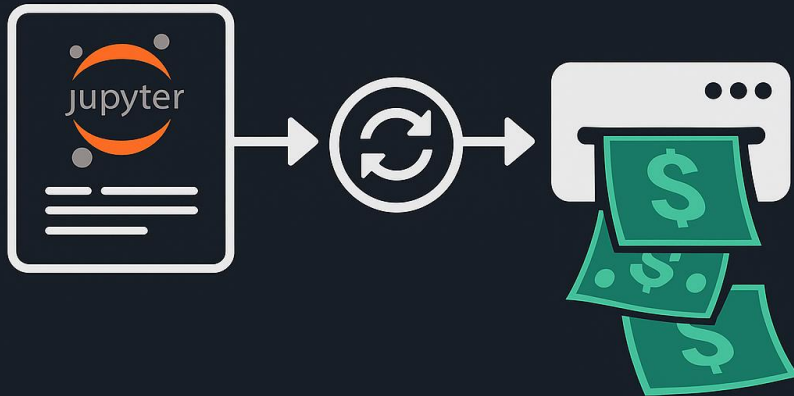
AI CARE

MONKEYUSER.COM



MLOps: Turning your Notebook into a Money-Printing Machine.

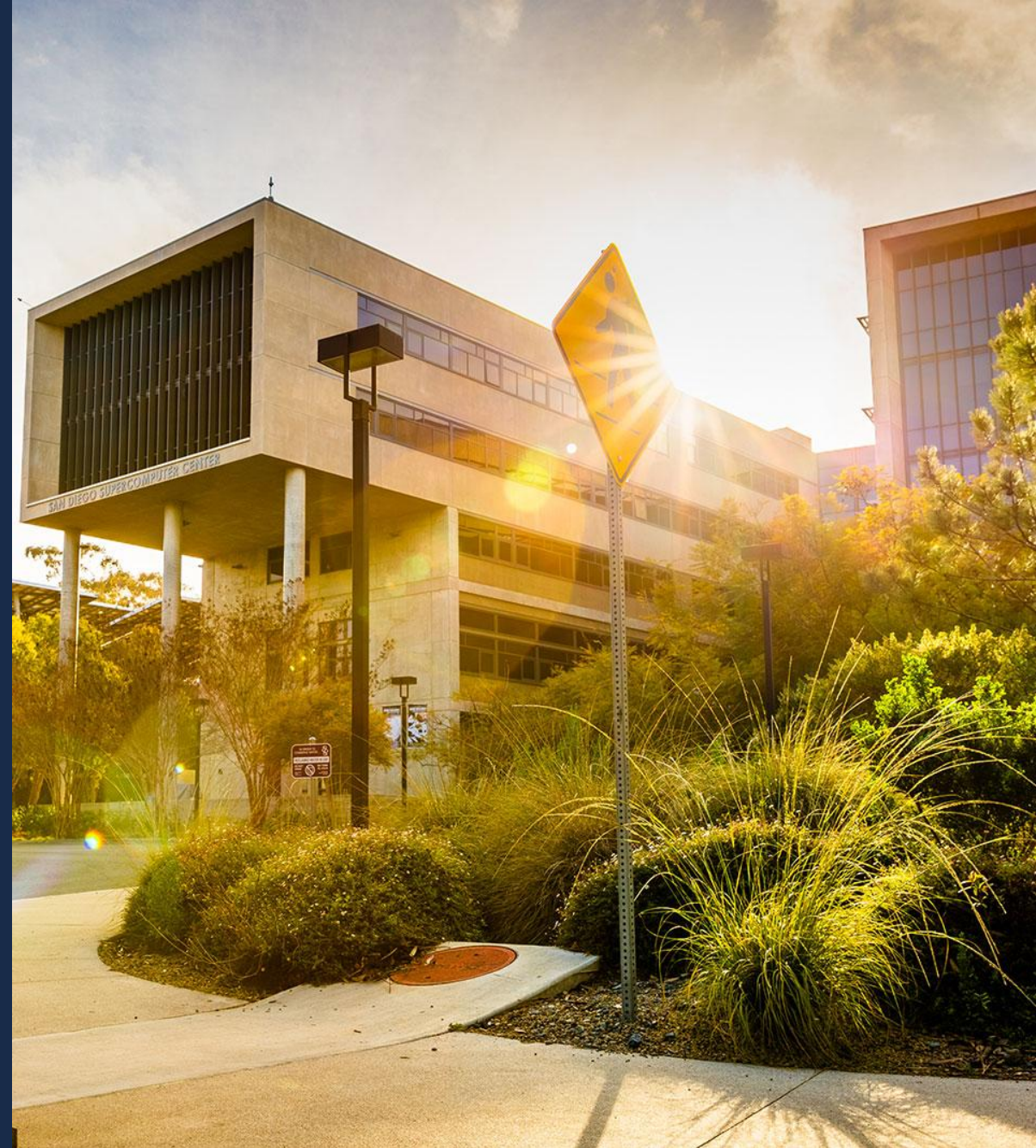
MLOps: Turning your Notebook into a Money-Printing Machine



1. Amazon: 136k Deps. a day.
2. Google: $O(10k)$ Deps a day.
3. Netflix: 500 Deps. a day.
4. Etsy: 50 Deps. a day.
5. Meta: 1 Dep. a day.

Final Remarks

1. There are bad ML/AI Engineers, and then there are ML/AI Engineers who can deploy their code to prod.
2. Don't commit big resources and time into testing ML/AI models. Use automated Unit Test first.
3. Github Actions are the 1st step in the automation process but there is plenty more you can learn to leverage your computing practices: (CI/CD, DevOps, MLOps).
4. AI won't take your AI Engineering job. Someone who knows how to use DevOps pipelines to streamline the deployment of AI/ML models into production and generates absurd amounts of revenue, that person will.





Questions



[linkedin.com/in/fgarzonm](https://www.linkedin.com/in/fgarzonm)



github.com/ofgarzon2662